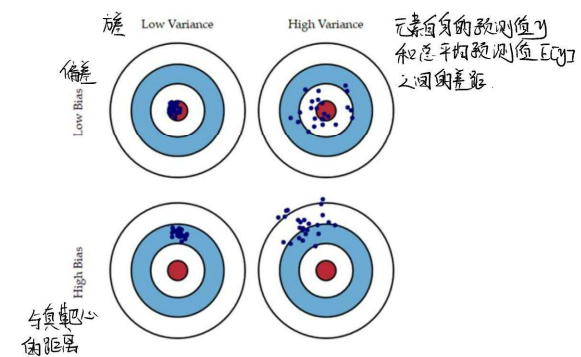




INT305 Machine Learning
Lecture 8
Bagging & Boosting

Siyue Yu
Department Intelligence Science
siyue.yu02@xjtlu.edu.cn

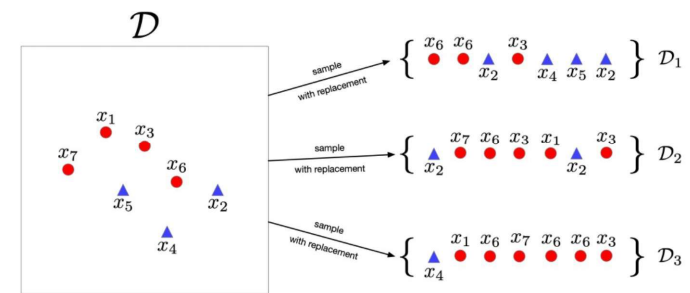
Bias/Variance Decomposition: Another Visualization



Bagging: The Idea

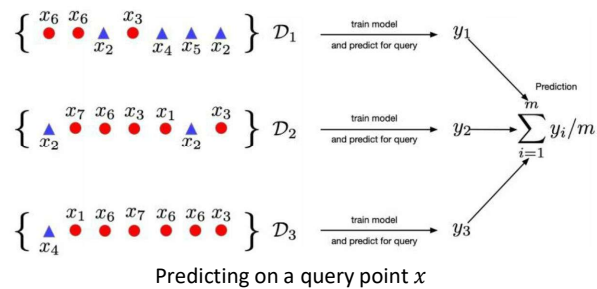
- In practice, the sampling distribution p_{sample} is often finite or expensive to sample from.
- So training separate models on independently sampled datasets is very wasteful of data!
 - Why not train a single model on the union of all sampled datasets?
- Solution: given training set $\mathcal{D}$, use the empirical distribution $p_{\mathcal{D}}$ as a proxy for p_{sample} . This is called **bootstrap aggregation**, or **bagging**.
 - Take a single dataset $\mathcal{D}$ with n examples.
 - Generate m new datasets ("resamples" or "bootstrap samples"), each by sampling n training examples from $\mathcal{D}$, with replacement.
 - Average the predictions of models trained on each of these datasets.
- The bootstrap is one of the most important ideas in all of statistics!
 - Intuition: As $|\mathcal{D}| \rightarrow \infty$, we have $p_{\mathcal{D}} \rightarrow p$

Bagging



In this example $n = 7, m = 3$

Bagging



Bagging: Effect on Hypothesis Space

- We saw that in case of squared error, bagging does not affect bias. 理论上, bagging 的期望偏差 E[bias] 不变
- But it can change the hypothesis space / inductive bias. 实践中, bagging 在经过针对全部模型的结果计算平均之后, 可以创造出一个不在个体模型得到的结果 (hypothesis space) 假设空间中的更好的结果。
- Illustrative example: 假如有一个非常受限的学习算法, 只有两种
 - $x \sim \mathcal{U}(-3, 3), t \sim \mathcal{N}(0, 1)$ 结果 $y = 1x$ 或 $y = -1x$.
 - $\mathcal{H} = \{wx | w \in \{-1, 1\}\}$ 这个算法是 高方差算法, 结果只在 1 和 -1 之间跳转.
- Sampled datasets & fitted hypotheses: 假如我们训练了 1000 个这样的不稳定模型 (假设每 500 个投了 1, 500 个投了 -1).



在计算平均之后 $y = \frac{500x + (-500x)}{1000} = 0$
- Ensembled hypotheses (mean over 1000 samples): y=0 不在算法提供的选项内



➤ The ensembled hypothesis is not in the original hypothesis space! 但 y=0 恰好是正确的。
- This effect is most pronounced when combining classifiers...

Bagging for Binary Classification

二分类方式

- If our classifiers output real-valued probabilities, $z_i \in [0, 1]$ then we can average the predictions before thresholding:

$$y_{\text{bagged}} = \mathbb{I}(z_{\text{bagged}} > 0.5) = \mathbb{I}\left(\sum_{i=1}^m \frac{z_i}{m} > 0.5\right)$$

- If our classifiers output binary decisions, $y_i \in \{0, 1\}$, we can still average the predictions before thresholding:

$$y_{\text{bagged}} = \mathbb{I}\left(\sum_{i=1}^m \frac{y_i}{m} > 0.5\right)$$

- A bagged classifier can be stronger than the average underlying model.
 - E.g., individual accuracy on "Who Wants to be a Millionaire" is only so-so, but "Ask the Audience" is quite effective.

方案一：软投票 (soft voting)

如果模型输出的结果是概率，比如模型1认为70%是A，模型2认为60%是A，模型3认为20%是A，那么就先算平均概率，再决定。

平均自信度 $z_{\text{bagged}} = (0.7 + 0.6 + 0.2) / 3 = 0.5$ ，阈值是0.5，0.5不大于0.5，所以集体自信度对于结果为A不足，所以预测结果为B。

方案二：硬投票 (hard voting) 少数服从多数

A=1 B=2

如果模型输出的就是它们的预测结果，比如模型1输出A，模型2输出A，模型3输出B。

对结果求平均：(1+1+0)/3 = 0.66，阈值0.5，0.66 > 0.5。

投票数大于50%，所以结果为A。

Bagging: Effect of Correlation

理想情况下如果有 m 个完全独立的训练集，模型从被降维的数据中，但现实是用 bagging，从 D 中创建 m 个 bootstrap datasets.

这些 bootstrap datasets 不是完全独立的，在 m 个高度相似的数据集上训练出来的 m 个模型，它们的预测也在高度相似 (correlation)，方差降低得以实现。

- Problem: the datasets are not independent, so we don't get the $1/m$ variance reduction.

➤ Possible to show that if the sampled predictions have variance σ^2 and correlation ρ , then

更真实的方差公式：前两项 ρ 相关。

$$\sigma^2: \text{个体模型的方差} \quad \rho: \text{相关系数，衡量了 } m \text{ 个 bagging 模型之间有多相关。}$$

$$\text{Var}\left(\frac{1}{m} \sum_{i=1}^m y_i\right) = \frac{1}{m} (1 - \rho) \sigma^2 + \rho \sigma^2$$

$$\text{Var}(\text{集内}) = \frac{1}{m} (1 - \rho) \sigma^2 + \rho \sigma^2$$

$\frac{1}{m} (1 - \rho) \sigma^2$: 对被消除的方差部分，增加模型数量 m ，这一项趋近于 0。

$\rho \sigma^2$: 无法消除多少模型，这一项也不会消失，这就是 bagging 集内无法超越的方差下限。

$\rho=0$ 代表个体模型之间完全不相关。

$\rho=1$ 代表个体模型之间非常雷同，1000 个模型训练的平均结果和 1 个模型没区别。

Random Forests

在构建每一棵决策树的每一个节点时，我们不再允许它看所有的特征（比如 width 和 height）。我们强迫它只能从随机抽取的一小部分特征中做选择！这就像把决策树的“眼睛”蒙住一部分。有些树只能看 width，有些树只能看 height。这样训练出来的树，肯定长得各不相同！

- Random forests = bagged decision trees, with one extra trick to decorrelate the predictions
 - When choosing each node of the decision tree, choose a random set of d input features, and only consider splits on those features
- Random forests are probably the best black-box machine learning algorithm – they often work well with no tuning whatsoever.
 - one of the most widely used algorithms in Kaggle competitions

BaggingSummary

- Bagging reduces overfitting by averaging predictions. (减少过度拟合): 通过降低方差。
- Used in most competition winners 即使你已经有了一个很强的模型, 再搞几个模型来 Bagging 一下 (集成), 通常都能把分数再刷高一点。
 - Even if a single model is great, a small ensemble usually helps.
- Limitations:
 - Does not reduce bias in case of squared error. 无法减少偏差
 - There is still correlation between classifiers. 相关性
 - Random forest solution: Add more randomness.
 - Naive mixture (all members weighted equally). 所有成员占比一样, 这不利于学习
 - If members are very different (e.g., different algorithms, different data sources, etc.), we can often obtain better results by using a principled approach to weighted ensembling.
- Boosting, up next, can be viewed as an approach to weighted ensembling that strongly decorrelates ensemble members.

Boosting

- **Boosting**
 - Train classifiers sequentially, each time focusing on training examples that the previous ones got wrong. boosting 的方式, 让下一轮的分类器花更多的精力到前一轮错误的地方
 - The shifting focus strongly decorrelates their predictions.
- To focus on specific examples, boosting uses a **weighted training set**.

Bagging 的工作方式是 parallel 的, boosting 是 sequential 的, 样本加权机制。

Bagging: 请 10 个水平差不多的老师, 每人随机改一部分卷子, 最后平均分。

Boosting: 先请第 1 个老师改卷子。他改错的那些难题, 被加粗标记出来, 交给第 2 个更厉害的老师专门攻克。第 2 个老师还搞不定的, 交给第 3 个.....

Weighted Trainingset

通过改变样本权重来影响模型的训练目标

- The misclassification rate $\frac{1}{N} \sum_{n=1}^N \mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$ weights each training example equally. 这个公式意味着每一个训练样本对最终误差的贡献都是一样的（权重均为 $1/N$ ）。模型会平等地对待所有样本。
- Key idea: we can learn a classifier using different costs (aka weights) for examples.

➤ Classifier “tries harder” on examples with higher cost

我们可以给不同的训练样本分配不同的代价 (costs) 或 权重 (weights)。

- Change cost function:

$$\sum_{n=1}^N \frac{1}{N} \mathbb{I}[h(x^{(n)}) \neq t^{(n)}] \text{ becomes } \sum_{n=1}^N w^{(n)} \mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$$

如果赋予某些样本更高的权重，分类器在训练时为了最小化总代价，就必须优先保证这些高权重样本被分类正确。

- Usually require each $w^{(n)} > 0$ and $\sum_{n=1}^N w^{(n)} = 1$.

$w^{(n)}$ 是每个样本的权重，通常要求所有 $w^{(n)} > 0$ 且总和 $\sum w^{(n)} = 1$ 。

AdaBoost (Adaptive Boosting)

- We can now describe the [AdaBoost](#) algorithm.
- Given a base classifier, the key steps of AdaBoost are:
 - At each iteration, re-weight the training samples by assigning larger weights to samples (i.e., data points) that were classified incorrectly.
 - Train a new base classifier based on the re-weighted samples.
 - Add it to the ensemble of classifiers with an appropriate weight.
 - Repeat the process many times.
- Requirements for base classifier:
 - Needs to minimize weighted error.
 - Ensemble may get very large, so base classifier must be fast. It turns out that any so-called [weak learner / classifier](#) suffices.
- Individually, weak learners may have high bias (underfit). By making each classifier focus on previous mistakes, AdaBoost [reduces bias](#).

1. 算法步骤： AdaBoost 通过一个循环过程来构建分类器集合：

步骤1（重加权）：在每一轮迭代开始时，调整训练样本的权重。对于上一轮被分类错误的样本，增加它们的权重；对于分类正确的样本，降低权重。

步骤2（训练）：基于这些重新加权的样本，训练一个新的基分类器（Base Classifier）。这个分类器会把重点放在处理那些之前做错的样本上。

步骤3（集成）：将这个新训练的基分类器加入到分类器集合中。注意，每个基分类器本身也会被分配一个权重（表现越好的分类器，在最终决策中话语权越大）。

步骤4（重复）：重复上述过程多次，构建出一系列分类器。

2.对基分类器的要求：

基分类器必须支持最小化加权误差（即能处理上一张幻灯片提到的加权代价函数）。

由于集成模型通常包含很多个基分类器，所以基分类器必须计算速度快。

弱分类器(WeakLearner)即可满足要求：即那些准确率只需略高于随机猜测（>50%）的简单模型（例如只有一层深度的决策树，称为“决策树桩”）。

3.降低偏差(Reducing Bias):

单个弱分类器通常太简单，存在高偏差(High Bias)和 欠拟合(Underfitting)的问题。

Boosting的作用：通过迫使每一个新的分类器去修正前一个分类器的错误，AdaBoost 能够逐步逼近复杂的决策边界。因此，AdaBoost 的主要作用是降低偏差 (Bias)，将多个弱分类器组合成一个强分类器。

WeakLearner/Classifier 弱学习器

- (Informal) Weak learner is a learning algorithm that outputs a hypothesis (e.g., a classifier) that performs slightly better than chance, e.g., it predicts the correct label with probability 0.51 in binary label case.
- We are interested in weak learners that are *computationally* efficient.
 - Decision trees
 - Even simpler: **Decision Stump**: A decision tree with a single split

[Formal definition of weak learnability has quantifies such as “for any distribution over data” and the requirement that its guarantee holds only probabilistically.]

非正式定义(Informal Definition):

弱学习器是一种学习算法，它输出的假设（模型）只需要比随机猜测 (Chance) 稍微好一点点即可。

在二分类问题（比如抛硬币猜正反）中，随机猜测的准确率是 50% (0.5)。

因此，一个弱分类器只要能在新数据上达到 51% (0.51) 的准确率，就被认为是合格的。

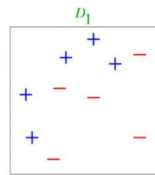
计算效率(Computationally Efficient):

我们不仅要求它能分类，还要求它计算速度非常快。因为 Boosting 需要串行训练成百上千个这样的分类器，如果单个分类器太慢，整体训练时间将不可接受。

首选模型:

1. 决策树 (Decision Trees): 特别是非常浅的树。
2. 决策树桩 (Decision Stump): 这是最简单的决策树，它只包含一个分裂节点 (single split)，也就是只问一个问题（比如“ $x > 5$ 吗？”）就立即做决定。

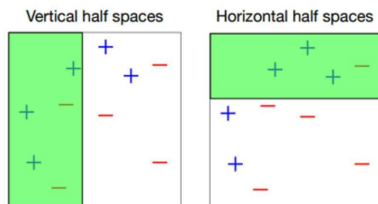
WeakClassifiers



左图(Vertical): 这是通过垂直切分 (例如 $x_1 < \text{threshold}$) 得到的分类边界。
右图(Horizontal): 这是通过水平切分 (例如 $x_2 < \text{threshold}$) 得到的分类边界。

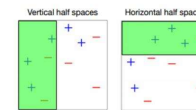
无论是垂直切还是水平切, 单一的决策树都无法完美地将所有 + 和 - 分开。它们只能做到“大概分对了一部分”, 这正符合“弱分类器”的定义。

These weak classifiers, which are decision stumps, consist of the set of horizontal and vertical half spaces.



WeakClassifiers

一个单独的弱分类器 (如决策树桩) 太简单了, 它无法使训练误差变得很小 (即高偏差 High Bias)。它无法拟合复杂的数据分布。



虽然单个分类器很弱, 但只要它保证比随机猜测好一点, 那么 AdaBoost 就可以通过组合它们, 最终逼近任意复杂的函数 (Universal Function Approximator)。AdaBoost 能把高偏差的弱分类器组合成一个低偏差的强模型。

- A *single* weak classifier is not capable of making the training error small
- But if can guarantee that it performs slightly better than chance, i.e., the weighted error of classifier h according to the given weights $\mathbf{w} \in \mathbf{w}, \dots$, w is at most $\frac{1}{2} - \gamma$ for some $\gamma > 0$, using it with AdaBoost gives us a universal function approximator!
- Last lecture we used information gain as the splitting criterion. When using decision stumps with AdaBoost we often use a “GINI Impurity”, which (roughly speaking) picks the split that directly minimizes error.
- Now let's see how AdaBoost combines a set of weak classifiers in order to make a better ensemble of classifiers...

在之前的决策树课程中, 我们使用“信息增益 (Information Gain)”来选择分裂点。

在 AdaBoost 中使用决策树桩时, 通常使用“基尼不纯度 (GINI Impurity)”作为标准。

简单来说, 基尼不纯度是一种更直接衡量分类误差的方法, 它能帮我们快速找到那个能让加权误差最小的切分点。

Notation in this lecture

- Input: Data $\mathcal{D}_N = \{\mathbf{x}^{(n)}, t^{(n)}\}_{n=1}^N$ where $t^{(n)} \in \{-1, +1\}$
 - This is different from previous lectures where we had $t^{(n)} \in \{0, 1\}$
 - It is for notational convenience, otw equivalent. 标签变化, = 分类的标签通常是 $t^{(n)} \in \{0, 1\}$ 在 AdaBoost 的推导中, 标签变成了 $t^{(n)} \in \{-1, +1\}$ (本质上两者是等价的)。
- A classifier or hypothesis $h: \mathbf{x} \rightarrow \{-1, +1\}$
 - loss: $\mathbb{I}[h(\mathbf{x}^{(n)}) \neq t^{(n)}]$ 分类器 $h(\mathbf{x})$ 的输出也相应地变为 $\{-1, +1\}$ 。

如果预测正确: h 和 t 符号相同, 乘积为 1。公式变为 $\frac{1}{2}(1-1)=0$

如果预测错误: h 和 t 符号相反, 乘积为 -1, 公式变为 $\frac{1}{2}(1-(-1))=1$ (有损失)。

AdaBoost Algorithm

- Input: Data $\mathcal{D}_N$, weak classifier WeakLearn (a classification procedure that returns a classifier h , e.g. best decision stump, from a set of classifiers $\mathcal{H}$, e.g., all possible decision stumps), number of iterations T
- Output: classifier $H(\mathbf{x})$
- Initialize sample weights: $w^{(n)} = \frac{1}{N}$ for $n = 1, \dots, N$
- For $t = 1, \dots, T$:
 - Fit a classifier to weighted data ($h \leftarrow \text{WeakLearn}(\mathcal{D}, \mathbf{w})$), e.g.,

$$h_t \leftarrow \underset{h \in \mathcal{H}}{\operatorname{argmin}} \sum_{n=1}^N w^{(n)} \mathbb{I}[h(\mathbf{x}^{(n)}) \neq t^{(n)}]$$
 - Compute weighted error $\text{err}_t = \frac{\sum_{n=1}^N w^{(n)} \mathbb{I}[h_t(\mathbf{x}^{(n)}) \neq t^{(n)}]}{\sum_{n=1}^N w^{(n)}}$
 - Compute classifier coefficient $\alpha_t = \frac{1}{2} \log \frac{1}{\text{err}_t}$ (∞)
 - Update data weights

$$w^{(n)} \leftarrow w^{(n)} \exp(-\alpha_t t^{(n)} h_t(\mathbf{x}^{(n)})) \left[\equiv w^{(n)} \exp(2\alpha_t \mathbb{I}[h_t(\mathbf{x}^{(n)}) \neq t^{(n)}]) \right]$$
- Return $H(\mathbf{x}) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}))$

输入与初始化:

- 输入数据 $\mathcal{D}_N$ 、弱学习算法 WeakLearn、以及迭代次数 T 。
- 初始化样本权重: 最开始, 所有 N 个样本的权重 $w_i^{(0)}$ 都是平等的, 设为 $1/N$ 。

迭代过程 (For $t = 1, \dots, T$):

算法进行了 轮循环, 每一轮执行以下 4 步:

1. 训练弱分类器 (Fit a classifier):

- 使用当前的加权数据训练一个弱分类器 h_t 。
- 目标是找到一个能使加权误差 $\sum_{i=1}^N w_i^{(t-1)} \mathbb{I}[h_t(x^{(i)}) \neq t^{(i)}]$ 最小化的分类器。这意味着模型会重点关注那些权重 $w_i^{(t-1)}$ 大的样本。

2. 计算加权误差率 (Compute weighted error err_t):

- 统计当前分类器 h_t 在所有样本上的加权误差率。
- 公式为: 所有被分错样本的权重之和 / 所有样本权重之和。

3. 计算分类器权重 (Compute classifier coefficient α_t):

- 这一步决定了当前这个分类器 h_t 在最终投票中有多少话语权。
- 公式: $\alpha_t = \frac{1}{2} \log \frac{1 - \text{err}_t}{\text{err}_t}$ 。

逻辑:

- 如果误差率 err_t 很小 (接近 0), α_t 就会很大 (正数), 说明这个分类器精准, 应该给它高权重。
- 如果误差率 err_t 接近 0.5 (类似随机乱猜), α_t 就会接近 0, 说明这个分类器没用。

4. 更新数据权重 (Update data weights):

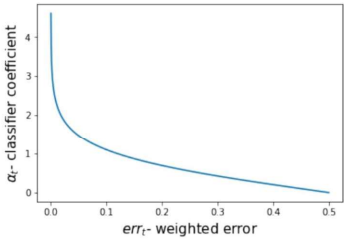
- 这是为下一轮训练做准备, 我们需要调整样本本权重 $w_i^{(t)}$ 。
- 公式: $w_i^{(t)} \leftarrow w_i^{(t-1)} \exp(-\alpha_t \mathbb{I}[h_t(x^{(i)}) \neq t^{(i)}])$ 。
- 核心机制:
 - 分类正确 ($t \cdot h_t = 1$): 权重乘以 $e^{-\alpha}$ (小于 1), 权重降低, 模型“不再关注”这些已经学会的样本。
 - 分类错误 ($t \cdot h_t = -1$): 权重乘以 e^{α} (大于 1), 权重增加, 模型在下一轮会“聚焦”关注这些错样本。

最终输出 (Return):

- 最终的强分类器 $H(x)$ 是有了 个弱分类器的加权组合。
- 公式: $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$ 。
- 当模型把每个弱分类器的投票结果乘以它的系数 α_t 然后求和, 如果结果大于 0, 预测 +1, 否则预测 -1。

WeightingIntuition

Recall: $H_t(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$ where $\alpha_t = \frac{1}{2} \log \frac{1 - \text{err}_t}{\text{err}_t}$



- Weak classifiers which get lower weighted error get more weight in the final classifier
- Also: $w_t^{(n)} \propto \exp(-2\alpha_t \mathbb{I}[h_t(x^{(n)}) \neq t^{(n)}])$
 - If err_t is high so α_t is low so misclassified examples get more attention
 - If err_t is low so α_t is high so misclassified examples are not

这张图解释了 AdaBoost 中两个关键的“权重”是如何变化的：分类器的权重 (α_t) 和 样本的权重 ($w^{(n)}$)。

1. 分类器权重 α_t (Classifier Coefficient):

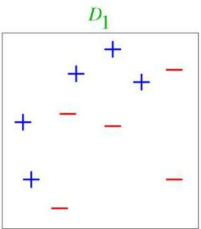
- 公式回顾: $\alpha_t = \frac{1}{2} \log \frac{1 - \text{err}_t}{\text{err}_t}$ 。
- 图表解读:
 - 横轴是误差率 err_t ，纵轴是权重 α_t 。
 - 曲线表明：当误差率 err_t 接近 0 时， α_t 非常大（甚至趋向无穷）。
 - 当误差率 err_t 接近 0.5 时， α_t 接近 0。
- 直觉：表现越好（误差越低）的弱分类器，在最终投票时的发言权（权重）就越大。如果一个分类器和瞎猜差不多（误差 0.5），它就没有发言权。

2. 样本权重 $w^{(n)}$ (Sample Weights):

- 公式回顾: $w^{(n)} \leftarrow w^{(n)} \exp(2\alpha_t \mathbb{I}\{h_t \neq t\})$ (这里稍微简化了形式，只看核心项)。
- 直觉:
 - 如果当前的弱分类器 h_t 非常准 ($\text{err}_t \approx 0$ ，导致 α_t 很高)：那么它犯错的那些极少数样本，一定是非常难搞的“硬骨头”。因此，这些错题的权重会大幅增加 (乘以一个很大的数)，迫使下一个分类器全力攻克它们。
 - 如果当前的弱分类器 h_t 很菜 ($\text{err}_t \approx 0.5$ ，导致 α_t 很低)：那么它犯错的样本可能只是普通难度，权重增加得不多。

AdaBoost Example

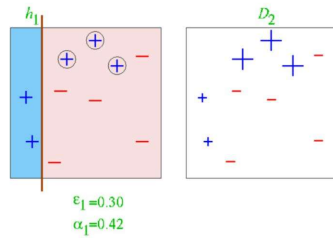
- Training data $\mathcal{a}$



[Slide credit: Verma & Thrun]

AdaBoost Example

• Round 1



$$\mathbf{w} = \left(\frac{1}{10}, \dots, \frac{1}{10} \right) \Rightarrow \text{Train a classifier (using } \mathbf{w}) \Rightarrow \text{err}_1 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_1(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^N w_i} = \frac{3}{10}$$

$$\Rightarrow \alpha_1 = \frac{1}{2} \log \frac{1 - \text{err}_1}{\text{err}_1} = \frac{1}{2} \log \left(\frac{1}{0.3} - 1 \right) \approx 0.42 \Rightarrow H(\mathbf{x}) = \text{sign}(\alpha_1 h_1(\mathbf{x}))$$

[Slide credit: Verma & Thrun]

初始状态: 有10个数据点, 所有点的权重
w相等 (1/10)

训练分类器 h_1 :

- 算法寻找一个最佳的垂直或水平切分线。
- 它选择了左侧的垂直线。
- 但右侧有3个蓝色点被错误地分类了。

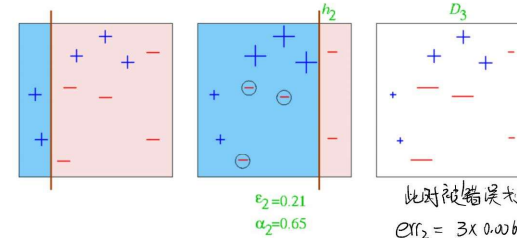
误差率 err_1 : 3个错点, 每个点权重0.1,
误差率是0.3

分类器权重 α_1 : $\frac{1}{2} \log \left(\frac{1-0.3}{0.3} \right) \approx 0.42$

下一轮这三个蓝点权重被放大。

AdaBoost Example

• Round 2



$$\mathbf{w} = \text{updated weights} \Rightarrow \text{Train a classifier (using } \mathbf{w}) \Rightarrow \text{err}_2 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_2(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^N w_i} = 0.21$$

$$\Rightarrow \alpha_2 = \frac{1}{2} \log \frac{1 - \text{err}_2}{\text{err}_2} = \frac{1}{2} \log \left(\frac{1}{0.21} - 1 \right) \approx 0.66 \Rightarrow H(\mathbf{x}) = \text{sign}(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}))$$

[Slide credit: Verma & Thrun]

重新计算10个点的权重: $W_{\text{new}} = W_{\text{old}} \times e^{\alpha_1} = 0.1 \times e^{0.42} \approx 0.15$ (错误的3个点)
 $W_{\text{new}} = W_{\text{old}} \times e^{-\alpha_1} = 0.1 \times e^{-0.42} \approx 0.065$ (正确的7个点)

基于加权后的数据进行
训练, 3个变大的蓝点很重要

为了正确划分这三个点, 选了
图2的分割线, 但这样就会
错误划分3个红色的点,

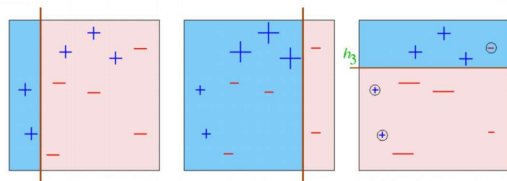
此时被错误划分的3个红点:

$$\text{err}_2 = 3 \times 0.069 \approx 3 \times 0.07 = 0.21$$

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1-0.21}{0.21} \right) = \frac{1}{2} \ln(3.76) \approx 0.5 \times 1.32 \approx 0.66$$

AdaBoost Example

Round 3



$$e_3 = 0.14$$

$$\alpha_3 = 0.92$$

$$e_{f_3} = 0.037 \times 4 \approx 0.14$$

$$\alpha_3 = \frac{1}{2} \ln \left(\frac{1 - 0.14}{0.14} \right) = \frac{1}{2} \ln(6.14)$$

$$\mathbf{w} = \text{updated weights} \Rightarrow \text{Train a classifier (using } \mathbf{w}) \Rightarrow \text{err}_3 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_3(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^N w_i} = 0.14 \approx 0.5 \times 0.81$$

$$\Rightarrow \alpha_3 = \frac{1}{2} \log \frac{1 - \text{err}_3}{\text{err}_3} = \frac{1}{2} \log \left(\frac{1}{0.14} - 1 \right) \approx 0.91 \Rightarrow H(\mathbf{x}) = \text{sign}(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}) + \alpha_3 h_3(\mathbf{x})) \approx 0.905$$

[Slide credit: Verma & Thrun]

AdaBoost Example

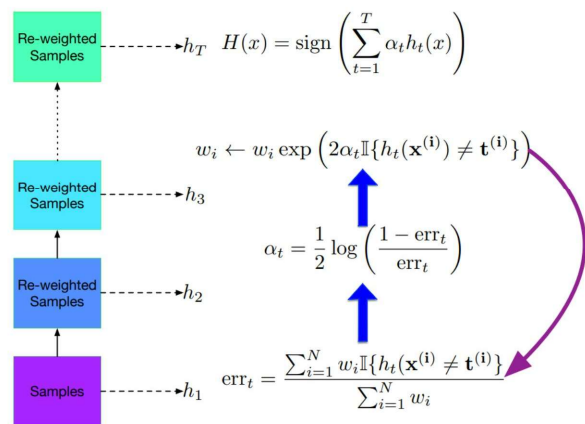
Final classifier

$$H_{\text{final}} = \text{sign} \left(0.42 \begin{array}{|c|} \hline \text{Plot 1} \\ \hline \end{array} + 0.65 \begin{array}{|c|} \hline \text{Plot 2} \\ \hline \end{array} + 0.92 \begin{array}{|c|} \hline \text{Plot 3} \\ \hline \end{array} \right)$$

$$= \begin{array}{|c|} \hline \text{Final Plot} \\ \hline \end{array}$$

[Slide credit: Verma & Thrun]

AdaBoost Algorithm



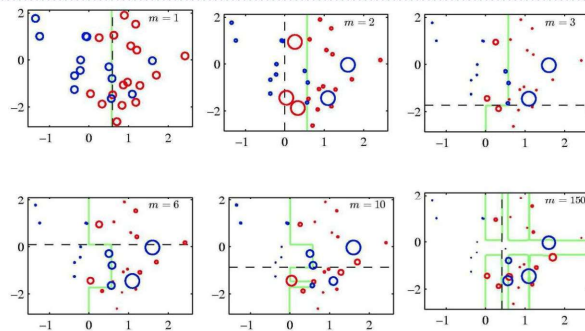
第二张幻灯片：AdaBoost 算法流程图 (Visualized Algorithm)

这张图用一个清晰的“流水线”展示了 AdaBoost 是如何一步步构建的。

1. 自底向上的构建过程：
 - 最底层 (紫色方块)：从原始的加权样本开始，训练第一个弱分类器 h_1 。
 - 计算误差 (箭头向右)：计算 h_1 的加权误差 err_1 。
 - 计算权重 (蓝色箭头向上)：根据误差计算 h_1 的话语权 α_1 。
 - 更新样本 (紫色箭头回环)：利用 α_1 和分类结果，计算新的样本权重 w_i 。分类错误的样本权重变大。
 - 下一层 (蓝色方块)：用重新加权 (Re-weighted) 的样本集，训练第二个弱分类器 h_2 。
2. 循环往复：
 - 这个过程不断重复：训练 -> 算权重 -> 重新加权样本 -> 训练下一个...
 - 直到训练出 T 个弱分类器 ($h_1, h_2, \dots, h_T$)。
3. 最终集成 (Top Right)：
 - 最终的强分类器 $H(x)$ 不是最新的那个 h_T ，而是所有历史分类器的加权总和： $\text{sign}(\sum \alpha_t h_t(x))$ 。

总结：
AdaBoost 就像一个循序渐进的学习过程。每一轮学习 (训练弱分类器) 都在为下一轮“划重点” (调整样本权重)。学得越好的老师 (弱分类器)，在划重点时越有权威 (α_t 大)，他指出的错题 (难样本) 在下一轮中就越受重视。最终的考试成绩由所有老师共同打

AdaBoost Example



- Each figure shows the number m of base learners trained so far, the decision of the most recent learner (dashed black), and the boundary of the ensemble (green)

AdaBoost Minimizes the Training Error

AdaBoost 算法的一个核心理论保证：它能以极快的速度最小化训练误差。

在每一轮迭代中，弱学习器 (WeakLearn) 产生的分类器 h_t ，其加权误差率 err_t 必须严格优于随机猜测。随机猜测的误差是 0.5， $err_t < 0.5 - \gamma$ (其中 $\gamma > 0$) 表示误差必须比 0.5 低一个固定量 γ ，这称为弱学习器的优势。

Theorem

Assume that at each iteration of AdaBoost the WeakLearn returns a hypothesis with error $err_t \leq \frac{1}{2} - \gamma$ for all $t = 1, \dots, T$ with $\gamma > 0$. The training error of the output hypothesis $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$ is at most

如果上述假设成立，那么经过 T 轮迭代之后，最终的强分类器 $H(x)$ 的训练误差 $L_N(H)$ 将会被限制在一个范围内。

$$L_N(H) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\{H(\mathbf{x}^{(i)}) \neq t^{(i)}\} \leq \exp(-2\gamma^2 T).$$

- This is under the simplifying assumption that each weak learner is γ -better than a random predictor.

$$L_N(H) \leq \exp(-2\gamma^2 T).$$

- This is called geometric convergence. It is fast!

$L_N(H)$: 最终模型的训练误差

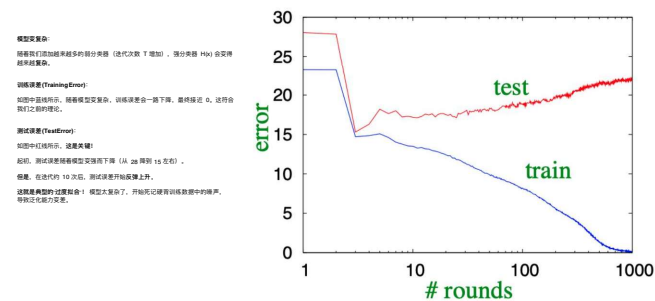
- 不得或近似 $\exp(-2\gamma^2 T) > 0$ 由于 γ 是常数，随 T 的增加，指数部分会变成非常小的数。
- 0 的无穷次方等于 0，这意味着训练误差趋近于 0。

γ (Gamma): 弱分类器优于随机猜测的程度。

T : 迭代次数 (弱分类器数量)

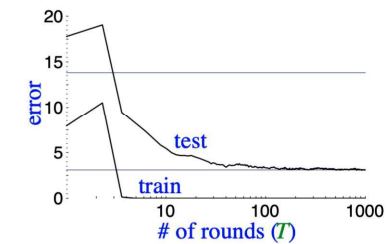
Generalization Error of AdaBoost

- AdaBoost's training error (loss) converges to zero. What about the test error of H ?
- As we add more weak classifiers, the overall classifier H becomes more "complex".
- We expect more complex classifiers overfit.
- If one runs AdaBoost long enough, it can in fact overfit.



Generalization Error of AdaBoost

- But often it does not!
- Sometimes the test error decreases even after the training error is zero!



观察结论:
这里的测试误差 (Test Error) 并没有像上一张图那样反弹上升。
相反, 它在训练误差降到 0 之后, 居然还在继续下降 (或者至少保持平稳, 没有恶化) !
即使迭代到了 1000 次, 模型已经极其复杂了, 它依然表现良好。

为什么会这样? (How does it happen?)
按照传统理论, 训练误差为 0 后, 再训练就是纯粹的“死记硬背”, 应该导致过拟合才对。
但 AdaBoost 似乎有一种“免疫力”, 即使在那之后, 它似乎还在学习某种“有用的东西”, 而不是噪声。

问题的状态:
S.D. 只最后说: “Next, we provide an alternative viewpoint on AdaBoost.”
这揭示了 AdaBoost 背后还有一个更深层的机制 (通常被称为 **间隔理论 (Margin Theory)**)。它解释了为什么 AdaBoost 在过拟合风险面前, 能够保持模型泛化能力的情况。简单来说, 即使所有点都分对了, AdaBoost 还在努力让分对的“信心 (Margin)”变得更大、更稳固。

- How does that happen?
- Next, we provide an alternative viewpoint on AdaBoost.

Additive Models

将 AdaBoost 解释为一种加法模型 (Additive Model) 的拟合过程

我们在 AdaBoost 中使用的弱分类器, 在这里被称为基函数, 它们代表假设空间.

Next, we'll now interpret AdaBoost as a way of fitting an additive model.

- Consider a hypothesis class $\mathcal{H}$ with each $h \in \mathcal{H}$ mapping $\mathbf{x}$ to $\{-1, +1\}$ within $\mathcal{H}$, i.e. are the "weak learners". And in this context they're also called **bases**.
- An **additive model** with m terms is given by

加法模型: 将多个基函数进行线性组合.

$$H_m(\mathbf{x}) = \sum_{i=1}^m \alpha_i h_i(\mathbf{x})$$

$H(\mathbf{x})$: 最终强模型
 $h_i(\mathbf{x})$: 第 i 个弱分类器的预测结果
 α_i : 第 i 个基函数的权重系数

where $(\alpha_1, \dots, \alpha_m) \in \mathbb{R}^m$.

- Observe that we're taking a linear combination of base classifiers $h_i(\mathbf{x})$, just like in boosting.
- Note also the connection to feature maps (or basis expansions) that we saw in linear regression and neural networks!

传统线性模型: 直接对原始特征 $\mathbf{x}$ 进行加权求和 ($W \cdot \mathbf{x}$)

加法模型 (AdaBoost): 先通过函数 h_i 将原始数据 $\mathbf{x}$ 转换 (映射) 成一个新的 "特征" (即分类结果 -1 或 $+1$), 然后再对这些 "新特征" 进行加权求和.

核心逻辑的变化:

Stagewise Training of Additive Models

A greedy approach to fitting additive models, known as **stagewise training**:

- Initialize $H_0(\mathbf{x}) = 0$
- For $m = 1$ to T :
 - Compute the m -th hypothesis $h_m = \argmin_{h \in \mathcal{H}} \sum_{i=1}^m \mathcal{L}(H_{m-1}(\mathbf{x}^{(i)}) + \alpha h(\mathbf{x}^{(i)}), t^{(i)})$ and α , assuming additive model $H_m = H_{m-1} + \alpha h_m$ is fixed:

$$(h_m, \alpha_m) \leftarrow \argmin_{h \in \mathcal{H}, \alpha} \sum_{i=1}^N \mathcal{L}(H_{m-1}(\mathbf{x}^{(i)}) + \alpha h(\mathbf{x}^{(i)}), t^{(i)})$$

- Add it to the additive model

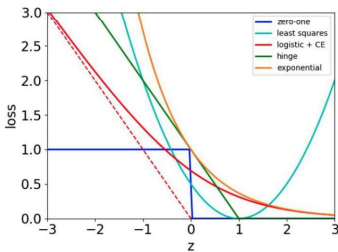
$$H_m = H_{m-1} + \alpha_m h_m$$

Additive Models with Exponential Loss

Consider the exponential loss

$$L_{\text{Exp}}(z) = \exp(-tz)$$

We want to see how the stagewise training of additive models can be done.



第一张幻灯片：指数损失与加法模型 (Exponential Loss)

这张幻灯片引入了一个关键的损失函数，并展示了它与其他损失函数的关系。

1. 指数损失函数 (Exponential Loss)

公式: $L_{\text{Exp}}(z) = \exp(-tz)$

- 其中 $t \in [-1, +1]$ 是真实标签, z 是模型的预测分数。
- 含义:
 - 如果预测正确 (t 和 z 同号, 乘积为正), $-tz$ 为负, $\exp(-tz)$ 会是一个很小的数 (损失小)。
 - 如果预测错误 (t 和 z 异号, 乘积为负), $-tz$ 为正, $\exp(-tz)$ 会呈指数级增长 (损失巨大)。

2. 图表对比 (Chart Comparison)

- 图中展示了多种损失函数的曲线 (z 为横轴, $t = 1$)。
- 青色线 (exponential): 这就是指数损失, 你会发现它不仅惩罚了 $z < 0$ 的错误 (左侧), 而且随着错误程度增加, 惩罚力量极其巨大 (指数级)。
- 蓝色线 (zero-one): 这是理想的 0-1 损失。
- 结论: 指数损失是 0-1 损失的一个上界 (Upper Bound), 并且它是光滑可导的, 这使得它非常适合用于优化。

第二张幻灯片：逐步训练 (Stage-wise Training)

这张图描述了一种如何用一种“贪心”的方法来训练加法模型, 这正是 Adaboost 的核心逻辑。

1. 核心理念 (Greedy Approach)

- 我们不想一次性算出所有了个基函数和系数, 那太难了。
- 我们采用逐步 (Stage-wise) 策略: 每次只训练一个基函数, 把它加进去, 然后再训练下一个。

2. 具体步骤

- 初始化: 从零开始, $H_0(x) = 0$ 。
- 循环 (For $m = 1$ to T):
 - 假设之前的模型 H_{m-1} 已经训练好并固定了。
 - 我们现在的目标是找到最新的一个基函数 h_m 和它的系数 α_m 。
 - 优化目标: 最小化加上这个新成员后的总损失!
- 这意味着, h_m 的任务就是去弥补 H_{m-1} 剩下的错误。
- 更新: 找到最佳的 h_m 和 α_m 后, 更新模型: $H_m = H_{m-1} + \alpha_m h_m$ 。

Additive Models with Exponential Loss

Consider the exponential loss $\mathcal{L}_{\text{Exp}} = \frac{1}{N} \sum_{i=1}^N \exp(-t f(\mathbf{x}^{(i)}))$

We want to see how the stagewise training of additive models can be done.

$$\begin{aligned} (h_m, \alpha_m) &\leftarrow \operatorname{argmin}_{h \in \mathcal{H}, \alpha} \sum_{i=1}^N \exp \left(- \left[H_{m-1}(\mathbf{x}^{(i)}) + \alpha h(\mathbf{x}^{(i)}) \right] t^{(i)} \right) \\ &= \sum_{i=1}^N \exp \left(-H_{m-1}(\mathbf{x}^{(i)}) t^{(i)} \right) \exp \left(-\alpha h(\mathbf{x}^{(i)}) t^{(i)} \right) \\ &= \sum_{i=1}^N w_i^{(m)} \exp \left(-\alpha h(\mathbf{x}^{(i)}) t^{(i)} \right). \end{aligned}$$

Here we defined $w_i^{(m)} \triangleq \exp \left(-H_{m-1}(\mathbf{x}^{(i)}) t^{(i)} \right)$ (doesn't depend on h , α).

Additive Models with Exponential Loss

We want to solve the following minimization problem:

$$(h_m, \alpha_m) \leftarrow \operatorname{argmin}_{h \in \mathcal{H}, \alpha} \sum_{i=1}^N w_i^{(m)} \exp \left(-\alpha h(\mathbf{x}^{(i)}) t^{(i)} \right). \quad (1)$$

- Recall from Slide 23 that

$$w^{(n)} \exp \left(-\alpha_t h_t(\mathbf{x}^{(n)}) t^{(n)} \right) \propto w^{(n)} \exp \left(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\} \right)$$

- Thus, for h_m , the above minimization is equivalent to:

$$h_m \leftarrow \operatorname{argmin}_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} \exp \left(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\} \right)$$

$$= \operatorname{argmin}_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} \left(\exp \left(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\} \right) - 1 \right) \quad \triangleright \text{subtract } \sum w_i^{(m)}$$

$$= \operatorname{argmin}_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\} \quad \triangleright \text{divide by } (\exp(2\alpha_t) - 1)$$

- This means that h_m is the minimizer of the weighted 0/1-loss.

Additive Models with Exponential Loss

- Now that we obtained h_m , we can plug it into our exponential loss objective (1) and solve for α_m .
- The derivation is a bit laborious and doesn't provide additional insight, so we skip it.
- We arrive at:

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right),$$

where err_m is the weighted classification error:

$$\text{err}_m = \frac{\sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h_m(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^N w_i^{(m)}}.$$

Additive Models with Exponential Loss

We can now find the updated weights for the next iteration:

$$\begin{aligned} w_i^{(m+1)} &= \exp \left(-H_m(\mathbf{x}^{(i)}) t^{(i)} \right) \\ &= \exp \left(- \left[H_{m-1}(\mathbf{x}^{(i)}) + \alpha_m h_m(\mathbf{x}^{(i)}) \right] t^{(i)} \right) \\ &= \exp \left(-H_{m-1}(\mathbf{x}^{(i)}) t^{(i)} \right) \exp \left(-\alpha_m h_m(\mathbf{x}^{(i)}) t^{(i)} \right) \\ &= w_i^{(m)} \exp \left(-\alpha_m h_m(\mathbf{x}^{(i)}) t^{(i)} \right) \end{aligned}$$

Additive Models with Exponential Loss

To summarize, we obtain the additive model $H_m(x) = \sum_{i=1}^m \alpha_i h_i(x)$ with

$$h_m \leftarrow \operatorname{argmin}_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h(\mathbf{x}^{(i)}) \neq t^{(i)}\},$$
$$\alpha = \frac{1}{2} \log \left(\frac{1 - \operatorname{err}_m}{\operatorname{err}_m} \right), \quad \text{where } \operatorname{err}_m = \frac{\sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h_m(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^N w_i^{(m)}},$$
$$w_i^{(m+1)} = w_i^{(m)} \exp \left(-\alpha_m h_m(\mathbf{x}^{(i)}) t^{(i)} \right).$$

这里的err和之前的err本质相同。

We derived the AdaBoost algorithm!

公式步骤	核心概念 (Causality)	关键信息点 (The Punchline)
前提 (Slides 2 & 5)	目标设定：我们想用“逐步贪心”的方式，最小化指数损失 $L_E = \exp(-tz)$ 。	指数损失是光滑且适合优化的 0-1 损失上界。
Slide 2 (核心简化)	定义样本权重：将目标函数中的 $\exp(-H_{m-1}t)$ 部分定义为当前样本的权重 $w_i^{(m)}$ 。	$\exp(-H_{m-1}t)$ 衡量了上一个模型 H_{m-1} 在样本 i 上犯了多少错误。错误越多，指数越高，权重 $w_i^{(m)}$ 越大。
Slide 3 (求解 h_m)	将优化问题转化为“加权分类”：通过代数简化，证明最小化指数损失等价于最小化加权分类误差。	结论：算法不需要直接优化指数损失，它只需要找到一个能最小化加权 0/1 损失的弱分类器 h_m 即可。（这证明了我们算法中的训练步骤是正确的）
Slide 5 (求解 α_m)	求解分类器权重：在确定 h_m 后，再用 h_m 的误率 err_m 来确定它的最终贡献 α_m 。	结论： $\alpha_m = \frac{1}{2} \log \left(\frac{1 - \operatorname{err}_m}{\operatorname{err}_m} \right).$ 这证明了我们算法中计算 α 的公式，正是最小化整体损失所需要的。
Slide 4 (更新权重)	推导权重更新公式：推导出新的样本权重 $w_i^{(m+1)}$ 是如何从旧权重 $w_i^{(m)}$ 和最新分类器 h_m	结论： $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m t)$ 。这证明了我们算法中“惩罚错误、奖励对题”的权重更新公式

BoostingSummary

- Boosting reduces bias by generating an ensemble of weak classifiers.
- Each classifier is trained to reduce errors of previous ensemble.
- It is quite resilient to overfitting, though it can overfit.

EnsemblesRecap

- Ensembles combine classifiers to improve performance
- Boosting
 - Reduces bias
 - Increases variance (large ensemble can cause overfitting)
 - Sequential
 - High dependency between ensemble elements
- Bagging
 - Reduces variance (large ensemble can't cause overfitting)
 - Bias is not changed (much)
 - Parallel
 - Want to minimize correlation between ensemble elements.